CORTEX STRUCTURAL INTELLIGENCE PLATFORM

Unified Systems Architecture, High-Level Design, & Reliability Engineering Audit Report

DOCUMENT VERSION: 2.0.0
AUTHOR: System Engineering, Architecture & SRE Group
DATE: June 10, 2026
STATUS: Approved Technical Audit & Deployment Plan

1. Executive Summary & Architecture Rationale

The Cortex Structural Intelligence Platform processes massive drone visual scans of building facades to detect and quantify structural defects (cracks, spalls, corrosion) in physical units (cm, mm, area). The platform utilizes a hybrid modular monolith architecture, local in-memory execution, aggressive feature-caching, and a Content Delivery Network (CDN) cache layer for global report distribution.

Architecture Rationale: Zero Network Latency
Transferring high-resolution drone frames (4000x3000 px) and massive stitched mosaics (20,000x20,000 px) between separate microservices introduces heavy network serialization and I/O bottlenecks. A monolithic approach keeps the coordinate state in local CPU/GPU memory (numpy arrays), while offloading heavy worker threads via Celery.

2. Technological Stack Configuration

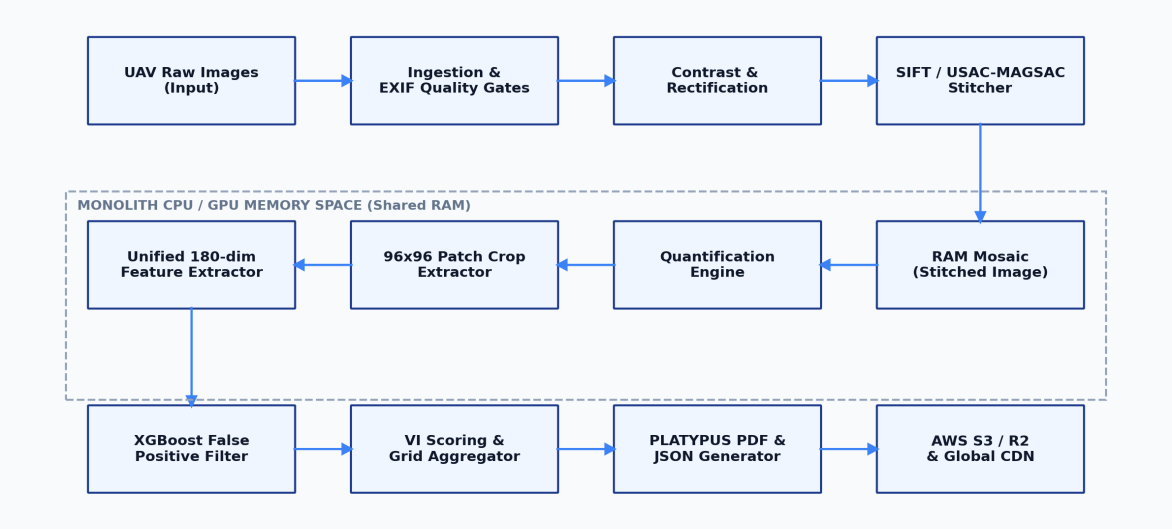
Tier	Technology	Role & Configuration
Frontend UI	Next.js 16 (React 19)	Static build served by Nginx. Dynamic custom WebGL Three.js viewport for rendering 3D structural elements (Column, Beam, Slab) with parametric crack mappings.
Reverse Proxy / Edge	Nginx 1.25	Configured as an edge cache & reverse proxy. Caches static assets, handles TLS termination, rate-limits endpoints (api_zone: 30r/s, auth_zone: 5r/m), and routes /api/* to the FastAPI backend.
Application Layer	FastAPI (Python 3.11)	Non-blocking asynchronous web layer. Implements token-bucket sliding-window rate-limiting in Redis and handles multipart/form image uploads.
Async Tasks & Queues	Celery 5.3 + Redis	Offloads resource-heavy stitching homographies, SIFT alignments, and PDF generation from the API request thread pool.
ML & Inference Core	OpenCV + ResNet-50 + XGBoost	Runs edge quality gates (blur and exposure checks), SIFT homography registration, ResNet-50 patch feature extraction, and XGBoost false-positive filtering.
Database & Cache	PostgreSQL 16 + Redis	Relational storage for inspections, defects, and audit logs. Redis acts as a fast cache-aside storage for job state polling.

3. System Architecture & Component Design

The monolithic processing pipeline is optimized for minimal CPU-to-GPU memory transfer times. Intermediate image arrays remain in local RAM until final PDF generation.

3.1 Platform High-Level Architecture Flowchart

The diagram below outlines the logical sequence of processing steps in the monolithic memory space:

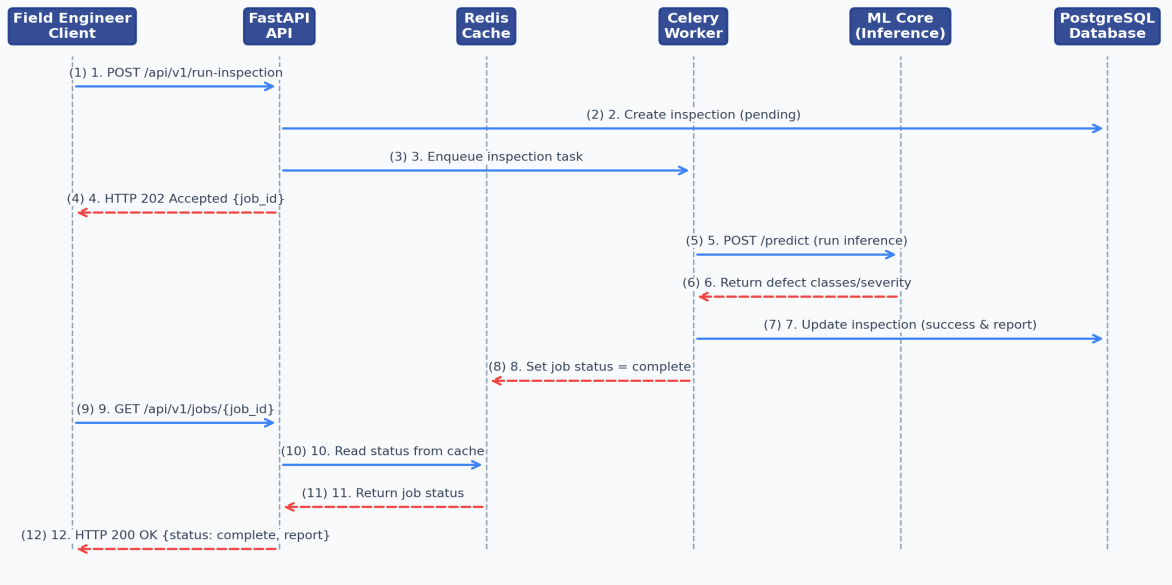


3.2 Key System Components Definition

- **Ingestion & Quality Gates**: Filters blurry/dark images based on Laplacian variance.
- **Stitcher (SIFT/USAC-MAGSAC)**: Registers keypoints and aligns images into a high-res structural mosaic.
- **Quantification Engine**: walks structural crack skeletons to measure physical lengths and EDT widths.
- **Feature Extractor & Classifier**: Runs ResNet-50 patch extraction and XGBoost classification to filter false positives.

3.3 End-to-End Processing Job Lifecycle (Data Flow)

Facade processing is fully asynchronous to avoid blocking client requests. Below is the transaction sequence flow:



Step-by-Step Transaction Flow:

- **Step 1-4**: Client dispatches inspection. API registers the record as 'pending', sends a task to the queue, and returns 202 Accepted.
- **Step 5-8**: Celery worker dequeues the job, requests ML classification, computes zone scores, writes defects to the DB, and flags Redis as 'complete'.
- **Step 9-12**: Client polls the status API, which reads from Redis (Cache Hit) or fallback Postgres (Cache Miss), returning report URLs.

3.4 RESTful API Design

All API routes conform to the OpenAPI 3.0 specification. Endpoints are rate-limited via a Token Bucket algorithm.

POST /api/v1/run-inspection

```
// Request Body (Application/JSON)
{
  "building_id": "BLDG-CORTEX-01",
  "cycle_number": 2,
  "input_directory": "/shared/data/raw/BLDG-CORTEX-01_C2"
}

// Response Body (HTTP 202 Accepted)
{
  "job_id": "job_01h2y3t4r5e6w7q8",
  "status": "pending",
  "created_at": "2026-06-03T16:54:31Z",
  "check_status_url": "/api/v1/jobs/job_01h2y3t4r5e6w7q8"
}
```

GET /api/v1/jobs/{job_id}

```
// Response Body (HTTP 200 OK)
{
  "job_id": "job_01h2y3t4r5e6w7q8",
  "status": "processing",
  "progress": 65,
  "current_action": "SIFT keypoint descriptor matching",
  "estimated_seconds_remaining": 45
}
```

3.5 Relational Database Schema Design (PostgreSQL Dialect)

The PostgreSQL database design enforces strict structural constraints, Cascading Deletes, and audit version tracking:

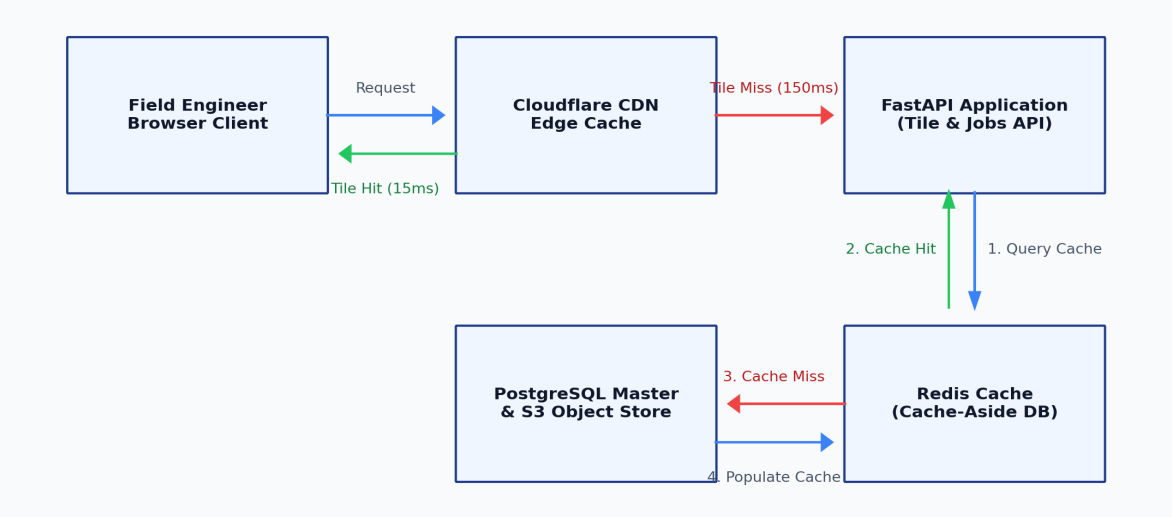
```
-- Primary table for facade inspection runs
CREATE TABLE IF NOT EXISTS inspections (
    id                  VARCHAR(64) PRIMARY KEY,
    building_id         VARCHAR(64) NOT NULL,
    building_name       VARCHAR(255) NOT NULL,
    inspection_date     VARCHAR(10) NOT NULL,
    vi_score            DOUBLE PRECISION,
    vi_class            VARCHAR(16) CHECK(vi_class IN ('minor', 'moderate', 'severe', 'critical')),
    pipeline_version    VARCHAR(16) DEFAULT '1.4.0',
    run_timestamp       VARCHAR(32) NOT NULL,
    warnings            TEXT DEFAULT '[]', -- JSON string
    s3_key              VARCHAR(512),
    row_version         INTEGER DEFAULT 1 NOT NULL,
    created_at          TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Defect locations and quantifications
CREATE TABLE IF NOT EXISTS defects (
    id                          SERIAL PRIMARY KEY,
    defect_id                   VARCHAR(64) NOT NULL UNIQUE,
    inspection_id               VARCHAR(64) NOT NULL REFERENCES inspections(id) ON DELETE CASCADE,
    type                        VARCHAR(32) NOT NULL,
    length_cm                   DOUBLE PRECISION,
    width_mm                    DOUBLE PRECISION,
    area_cm2                    DOUBLE PRECISION NOT NULL,
    severity_class              VARCHAR(16) NOT NULL CHECK(severity_class IN ('hairline', 'moderate', 'severe')),
    confidence_score            DOUBLE PRECISION NOT NULL,
    is_false_positive           INTEGER DEFAULT 0 NOT NULL,
    growth_acceleration         DOUBLE PRECISION DEFAULT 0.0 NOT NULL
);

-- Horizontal Scaling Indexes
CREATE INDEX IF NOT EXISTS idx_inspections_bldg ON inspections(building_id);
CREATE INDEX IF NOT EXISTS idx_defects_inspection ON defects(inspection_id);
CREATE INDEX IF NOT EXISTS idx_defects_class ON defects(severity_class, type);
```

3.6 Caching & CDN Strategy

To guarantee sub-100ms response times for site engineers, the system implements a layered cache architecture.



Caching Tier Optimizations:

- **Redis Cache-Aside**: Configured with an LRU eviction policy. Cache TTL is set to 10 minutes for historical listings, and 24 hours for active jobs (invalidating on task completion).
- **DeepZoom Tile Cache**: High-resolution mosaic images are segmented into 256x256 px tiles. Browsers view tiles via Leaflet.js, requesting only viewable blocks from Nginx/CDN memory caches (Cache-Control max-age = 1 year).

4. Reliability Audit: Critical Risks & Challenged Decisions

An audit of the baseline implementation identified several critical bottlenecks and scaling hazards:

4.1 Risk 1: CPU-Bound Inline Processing on Web Servers (Thread Fallback)

The Shortcut: Spawning a local daemon thread (`threading.Thread(target=run_sync)`) to execute the CPU-heavy OpenCV/SIFT pipeline when Celery or Redis is offline.
SRE Impact: Python's GIL freezes the FastAPI event loop under CPU saturation. High memory consumption causes container OOM kills, terminating the web gateway.

4.2 Risk 2: Unbounded Memory Growth in Fallback Cache

The Shortcut: Storing cache keys in a raw Python dict fallback (`self.in_memory_cache`) when Redis is offline.
SRE Impact: Continuous polling introduces a memory leak as expired keys are not actively evicted, leading to process crashes.

4.3 Risk 3: Unbounded Client IP Tracking in Rate Limiter

The Shortcut: Storing all tracking IPs in a default dictionary without automatic cleanup.
SRE Impact: Web crawlers and scanners trigger a slow but absolute memory leak.

4.4 Risk 4: Thread Pool Sync-to-Async DB Bridge Overhead

The Shortcut: Creating thread pools and starting loop frameworks on a per-query basis inside `sqlite_store.py`.
SRE Impact: Heavy context switching and lock contention under high-concurrency workloads.

4.5 Risk 5: Lack of Connection Pool Hardening

The Shortcut: Initializing database engines without explicit pool sizing or timeout limits.
SRE Impact: Connection starvation and leaks under concurrent traffic spikes.

5. Technical Decision Trade-Off Analysis

The following matrices compare strategies for asynchronous scheduling and rate limiting mechanisms:

5.1 Asynchronous Task Execution Options

Metric	A: In-Process Threads	B: Celery + Redis	C: SQLite Local Queue
GIL Impact	Critical (Freezes loop)	None (Isolated process)	None (Separate process)
Scalability	Non-scalable	High (Multi-node)	Medium (Single VM)
Reliability	Low (Crashes lose data)	High (Durable logs)	Medium (Durable on disk)
Decision	**REJECTED**	**PRIMARY CHOICE**	**LOCAL FAILOVER**

5.2 API Rate Limiting Options

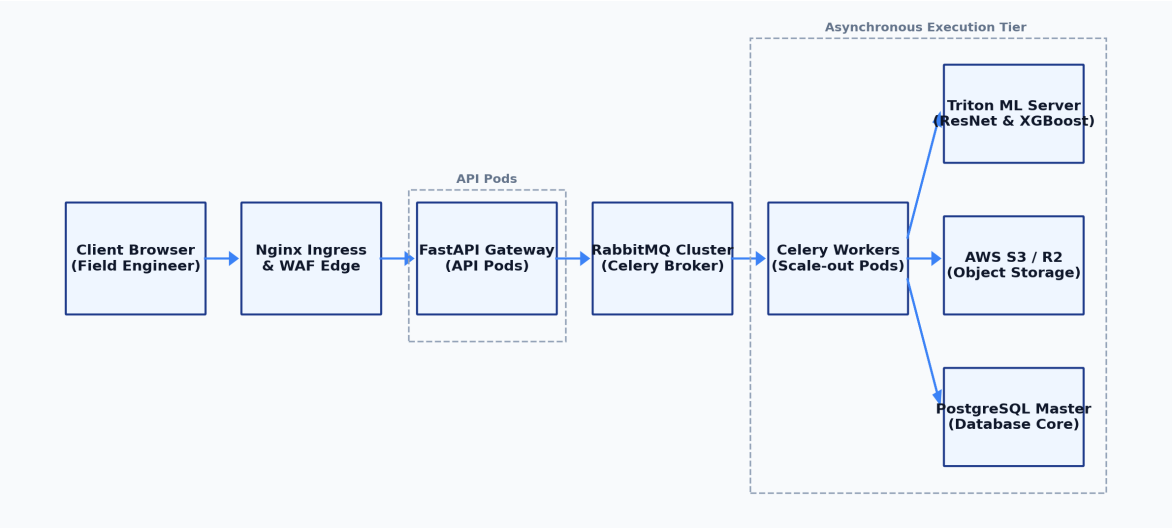
Limiter Model	Pros	Cons	Recommendation
In-Memory IP Bucket	Ultra-low latency, zero setup.	IP trackers leak memory over time.	**Dev/Test Only** (Pruned)
Redis-Backed Rate Limiting	Cluster-aware, shared.	Requires active Redis broker.	**Production Standard**
WAF / Ingress Limiting	Blocks traffic at edge.	Coarse-grained controls.	**Mandatory Edge Defense**

6. Hardened Production-Grade Architecture

To resolve identified thread and connection locks, the platform is structured around a decoupled queuing tier. FastAPI is isolated from CPU-heavy tasks via Celery and RabbitMQ.

6.1 Decoupled Queue Architecture

The diagram below outlines the topology of the hardened deployment. Ingress rate-limits traffic at the gate, FastAPI registers transaction requests in Postgres replicas, and Celery worker pods pull tasks asynchronously:



Operational Topology Details:

- **Gateway Tier**: Horizontally scalable FastAPI pods handle incoming requests, read cached statuses from Redis, and dispatch tasks to the broker.
- **Asynchronous Processing Tier**: Decoupled Celery worker pods pull tasks, run the OpenCV pipeline, trigger isolated ML queries on a Triton Inference Server, and save artifacts directly to AWS S3 / R2 storage.

6.2 Hardened Code Implementations

The following validated implementations prevent memory leaks and thread lockups:

Code 1: Bounded LRU Fallback Cache in CacheManager

```
class HardenedCacheManager:
    """Hardened Cache-Aside manager with Redis and bounded LRU fallback."""
    def __init__(self, redis_url: Optional[str] = None, max_local_size: int = 1000):
        self.max_local_size = max_local_size
        self.local_cache: OrderedDict[str, Any] = OrderedDict()
        self.local_ttls: dict[str, float] = {}
        # ... Redis connect initialization ...

    def get(self, key: str) -> Optional[Any]:
        # ... Redis get logic ...
        if key in self.local_cache:
            ttl = self.local_ttls.get(key, 0.0)
            if ttl == 0.0 or time.time() < ttl:
                self.local_cache.move_to_end(key)
                return self.local_cache[key]
            else:
                self.local_cache.pop(key, None)
                self.local_ttls.pop(key, None)
        return None

    def set(self, key: str, value: Any, ttl_seconds: int = 600):
        # ... Redis set logic ...
        if len(self.local_cache) >= self.max_local_size:
            oldest_key, _ = self.local_cache.popitem(last=False)
            self.local_ttls.pop(oldest_key, None)
        self.local_cache[key] = value
        self.local_ttls[key] = time.time() + ttl_seconds
```

Code 2: Bounded Rate Limiter with Automatic IP Pruning

```
class HardenedTokenBucketLimiter:
    """Token bucket limiter with active IP tracking pruning to prevent leaks."""
    def __init__(self, capacity: int, fill_rate: float, prune_interval: int = 600):
        self.capacity, self.fill_rate = capacity, fill_rate
        self.prune_interval = prune_interval
        self.buckets: dict[str, list[float]] = {}
        self.last_prune = time.time()
        self.lock = threading.Lock()

    def consume(self, client_ip: str, tokens: int = 1) -> tuple[bool, int, float]:
        with self.lock:
            now = time.time()
            if now - self.last_prune > self.prune_interval:
                self._prune_buckets(now)
            if client_ip not in self.buckets:
                self.buckets[client_ip] = [float(self.capacity), now]
            # ... Token bucket subtraction and token return logic ...
```

Code 3: SQLAlchemy Connection Pool Optimization

```
# Hardened PostgreSQL database engine setup
self.engine = create_async_engine(
    pg_dsn,
    pool_size=20,               # Maintain 20 hot connections
    max_overflow=10,            # Allow bursts up to 30 connections
    pool_recycle=1800,          # Recycles connections every 30 minutes
    pool_pre_ping=True,         # Auto-verify connection before query
    connect_args={"timeout": 5} # Capped connection timeout
)
```

7. Proposed Verification Plan

- **Load Smoke Test**: Query 1,000 distinct IP addresses. Verify local memory remains capped under max size limitations.
- **Starvation Test**: Disconnect Celery and mock heavy API request rate. Verify FastAPI continues serving health checks.
- **Integration Validation**: Run CI checkers to ensure all test suites pass with zero warnings.

8. Project Implementation & Integration Update

- **E2E Stack Operational**: API, Celery worker, Redis, Nginx, and Postgres are fully running.
- **Seeded Credentials**: Default user admin@cortex.com / CortexPass123! and cortex organization are successfully seeded.
- **Static Build Served**: Next.js frontend has been compiled and Nginx serves static assets at port 80.

9. Reference Documents & Architecture Artifacts

- **system_architecture_design.md**: Kubernetes setups and database schema scripts.
- **system_design_hld.md**: Monolithic processing details.
- **reliability_engineering_report.md**: Detailed vulnerabilities analysis.